

MOBILE APPLICATION AND DEVELOPMENT – WEEK 2

Git for Android Development

Repositories, commits and branches · GitHub · Version control inside Android Studio

Ekramul Haque Amin

Lecturer, Dept. of AI & Data Science

TODAY'S SESSION – THEORY 2

Agenda

01 Why version control, and Git's core mental model

02 Committing changes — status, add, commit, log, and .gitignore

03 Branching and merging, and resolving a conflict

04 GitHub, remotes, and Git inside Android Studio

FOLLOWING ON FROM THEORY 1

You Just Built Something Worth Protecting

This morning you built your first Android project. What happens when you break it tomorrow trying something new, and can't remember what you changed?

History

Every saved version, forever — undo anything, anytime

Collaboration

Multiple people editing the same project without overwriting each other

Safety

Try a risky change on a branch — the working version is never at risk

Git is the tool. GitHub is where a copy of your Git history can live online.

PART 1 – FUNDAMENTALS

What Is Git?

Git is a distributed version control system — a tool that snapshots your project's files over time, so you can look back, compare, and restore any earlier point.

Distributed

Every clone has the full history — not just the current files

Git ≠ GitHub

Git is the tool that runs on your machine; GitHub is one website that hosts Git repositories

Android Studio has Git built in (VCS menu), but understanding the underlying commands makes that UI make sense.

BEFORE YOUR FIRST COMMIT

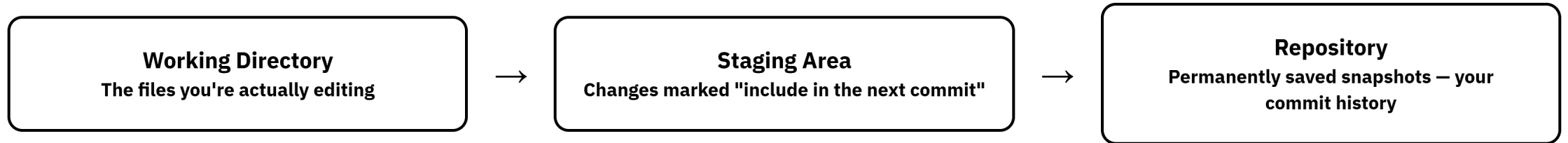
Setting Up Git

```
git --version           // check it's installed
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

This name and email are attached to every commit you make — set them once per computer, not once per project.

THE ONE IDEA THAT MAKES GIT CLICK

Three Places Your Code Can Be



`git add` moves changes from Working Directory to Staging. `git commit` moves Staging into the Repository, permanently.

TURNING A FOLDER INTO A REPOSITORY

git init

```
cd MyFirstApp  
git init
```

Creates a hidden `.git` folder — that's the entire database of your project's history. Delete it, and the folder is just files again, with no history.

Android Studio does this for you automatically if you check "Create Git repository" while making a new project.

NOT EVERYTHING BELONGS IN HISTORY

.gitignore for Android

Generated build output and machine-specific files should never be committed — they're huge, they change constantly, and Gradle regenerates them anyway.

```
*.iml  
.gradle/  
/local.properties  
/.idea/  
.DS_Store  
/build  
/captures  
.externalNativeBuild  
.cxx
```

Android Studio's "New Project" wizard generates a correct .gitignore automatically — check "Create Git repository" and it's already there.

THE EVERYDAY LOOP

status → add → commit

```
git status                // what's changed?  
git add MainActivity.java // stage one file  
git add .                 // stage everything changed  
git commit -m "Add login screen"
```

Run `git status` often — it's the safest command in Git, and it tells you exactly what will happen next.

A MESSAGE FUTURE-YOU WILL THANK YOU FOR

Writing Good Commit Messages

Weak

"fix"
"updates"
"asdf"

Better

"Fix crash on empty username field"
"Add SharedPreferences login flag"

Present tense, one clear change per commit, and specific enough that git log alone explains your project's history.

LOOKING AT WHAT HAPPENED

git log and git diff

```
git log --oneline      // one line per commit
git diff              // unstaged changes, line by line
git diff --staged     // what's about to be committed
```

git diff before every commit is a habit worth building now — it catches the debug `Log.d()` line or leftover comment before it ships.

PART 2 – BRANCHING

What Is a Branch?

A branch is an independent line of development — a way to work on something new without touching the version that already works.

main

The stable, working version of your project

feature/login-screen

A separate line where you build the new login screen

If the new feature breaks something, main is untouched. When it's ready, you merge the branch back in.

CREATING AND MOVING BETWEEN BRANCHES

branch, checkout, switch

```
git branch                // list branches
git checkout -b feature/login // create AND switch to it
git switch main           // move back to main
```

checkout -b is shorthand for "create this branch, then switch to it" — the single command most people reach for.

BRINGING THE WORK BACK TOGETHER

git merge

```
git switch main  
git merge feature/login
```

You merge **INTO** the branch you're currently on. Switch to main first, then merge the feature branch into it — not the other way around.

WHEN GIT CAN'T DECIDE FOR YOU

Resolving a Merge Conflict

Happens when two branches changed the SAME lines differently. Git marks both versions in the file and waits for you to choose.

```
<<<<<< HEAD
    android:text="Welcome!"
=====
    android:text="Hello there!"
>>>>>> feature/login
```

Edit the file to keep the version you want, delete the <<</==>>> markers, then `git add` and `git commit` to finish the merge.

ENOUGH STRUCTURE, NO MORE

A Simple Workflow for This Course

- 1 main always holds a working, buildable version of your project**
- 2 Every new feature or lab exercise gets its own short-lived branch**
- 3 Merge back to main once it works, then delete the branch**

PART 3 – GITHUB & REMOTES

Your Repo, in the Cloud

GitHub hosts a copy of your Git repository online — a "remote". It adds a backup, a way to share code, and the tools to collaborate: pull requests, issues, and code review.

Local repository

On your computer — what git init created

Remote repository

On GitHub — a shared copy everyone syncs with

GITHUB.COM → NEW REPOSITORY

Creating a Repository on GitHub

- 1** Give it a name — matching your Android project name is a good habit
- 2** Choose Public or Private
- 3** Leave it empty (no README) if you already have a local project to push

GitHub then shows you the exact commands to connect your existing local repo to it — copy them.

TWO WAYS TO CONNECT LOCAL AND REMOTE

clone vs. remote add

Starting from GitHub

```
git clone https://github.com/you/MyFirstApp.git
```

Downloads the whole repository, already connected

Starting from a local project

```
git remote add origin https://github.com/you/MyFirstApp.git
```

Links an existing local repo to an empty GitHub one

SYNCING LOCAL AND REMOTE

push, pull, and fetch

```
git push -u origin main    // send local commits to GitHub (first time)
git push                   // send local commits (after that)
git pull                   // download and merge remote changes
```

Pull before you push — if someone else's changes are on GitHub and you haven't pulled them, Git will refuse the push until you do.

EVERYTHING WE JUST TYPED, AS BUTTONS

Git Inside Android Studio

VCS → Commit

A checkbox list of changed files =
git add + git commit

Green up-arrow (Push)

= git push

Blue down-arrow (Update)

= git pull

Git → Branches

Create, switch, and merge from
a dropdown

VCS → Import into Version Control → Share Project on GitHub does the whole init + remote add + first push in one dialog.

PART 4 – WRAP-UP

Pull Requests & Code Review

A pull request (PR) asks to merge one branch into another on GitHub — it's a review checkpoint before code reaches main, not just a merge button.

Opening one

Push your feature branch, then GitHub shows "Compare & pull request" — describe what changed and why

Why it matters

A second pair of eyes catches bugs early and keeps main always in a working state

THE FIRST FILE ANYONE OPENS

README.md Basics

```
# MyFirstApp
```

```
A simple Android app that greets the user.
```

```
## Setup
```

1. Clone: `git clone https://github.com/you/MyFirstApp.git`
2. Open in Android Studio
3. Run on an emulator or device

GitHub renders this Markdown file automatically on your repo's homepage — it's what a teammate reads before touching your code.

LEARN FROM THESE NOW

Common Git Mistakes

Committing build folders

Forgetting `.gitignore` fills the repo with generated `/build` and `.gradle` files

Committing straight to main

Skips review and can leave main broken for everyone

Force-pushing (git push -f)

Overwrites remote history and can erase a teammate's commits

Forgetting to pull first

Leads to avoidable merge conflicts and a rejected push

IN-CLASS EXERCISE, 10 MINUTES

Put Your Hello World App on GitHub

- 1 In Android Studio, run `git init` on your Theory 1 project and add a `.gitignore`
- 2 `git add .` and `git commit -m "Initial commit"`
- 3 Create an empty repository on GitHub and connect it with `git remote add origin`
- 4 `git push -u origin main`, then create a feature/change-text branch, edit the `TextView`, commit, and merge it back into main

BEFORE YOU LEAVE TODAY

Recap Checklist

1. Can you explain the three states: working directory, staging area, repository?

2. Do you know the difference between git add, commit, push, and pull?

3. Can you create a branch, make a change, and merge it back into main?

4. Do you know what a pull request is for, beyond just merging code?

WHERE WE GO FROM HERE

Next Steps

This Week – Lab

Hands-on Git practice: init, commit, branch, and push your first app deployment to GitHub

Week 3 – Theory

Core Android Components: Activities, Fragments, and the Activity lifecycle

Keep committing as you build – small, frequent commits make every future assignment easier to track and undo.

Questions?

git init

commit

branch & merge

GitHub

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)